



PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 4

Database Management – SQL & PL/SQL

SESSION / ASSIGNMENT – 7

TASKS :

1. Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.

→ CODE:

```
CREATE TABLE Restaurant (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    location VARCHAR(100) NOT NULL,  
    cuisine VARCHAR(50)  
);
```

OUTPUT:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None			Change Drop More
☐	2 name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 location	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 cuisine	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More

2. Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).

➔ CODE:

```
CREATE TABLE FoodOrder (  
    order_id INT PRIMARY KEY,  
    restaurant_id INT,  
    user_id INT,  
    order_total DECIMAL(10,2),  
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
);
```

OUTPUT:

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	order_id 	int(11)			No	None			 Change  Drop  More
☐	2	restaurant_id 	int(11)			Yes	NULL			 Change  Drop  More
☐	3	user_id	int(11)			Yes	NULL			 Change  Drop  More
☐	4	order_total	decimal(10,2)			Yes	NULL			 Change  Drop  More

3. Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.

Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

➔ CODE: Table - SpotifyUser

```
CREATE TABLE SpotifyUser (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    email VARCHAR(100) NOT NULL,  
    subscription_type VARCHAR(30)  
);
```

OUTPUT:

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	user_id 	int(11)			No	None			 Change  Drop  More
☐	2	username 	varchar(50)	utf8mb4_general_ci		Yes	NULL			 Change  Drop  More
☐	3	email	varchar(100)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐	4	subscription_type	varchar(30)	utf8mb4_general_ci		Yes	NULL			 Change  Drop  More

CODE: Insert 6 users

```
INSERT INTO SpotifyUser
(user_id, username, email, subscription_type)
VALUES
(1, 'haariss', 'haaris@gmail.com', 'Premium'),
(2, 'john123', 'john@gmail.com', 'Free'),
(3, 'alex_music', 'alex@gmail.com', 'Premium'),
(4, 'sarah22', 'sarah@gmail.com', 'Student'),
(5, 'mike_rock', 'mike@gmail.com', 'Premium'),
(6, 'emma_play', 'emma@gmail.com', 'Free');
```

OUTPUT:

	user_id	username	email	subscription_type
☐ Edit Copy Delete	1	haariss	haaris@gmail.com	Premium
☐ Edit Copy Delete	2	john123	john@gmail.com	Free
☐ Edit Copy Delete	3	alex_music	alex@gmail.com	Premium
☐ Edit Copy Delete	4	sarah22	sarah@gmail.com	Student
☐ Edit Copy Delete	5	mike_rock	mike@gmail.com	Premium
☐ Edit Copy Delete	6	emma_play	emma@gmail.com	Free

4. Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

→ CODE:

```
CREATE VIEW TopSpendersView AS
SELECT
    SpotifyUser.username,
    FoodOrder.order_total
FROM SpotifyUser
JOIN FoodOrder
    ON SpotifyUser.user_id = FoodOrder.user_id
WHERE FoodOrder.order_total > 1000;
```

5. Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.

Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.

➔ **CODE:**

```
SELECT username, order_total
FROM TopSpendersView
WHERE order_total > 2000
ORDER BY order_total DESC;
```

OUTPUT:

Showing rows 0 - 1 (2 total, Query took 0.0005 seconds.) [order_total: 3200.00... - 2500.00...]

```
SELECT username, order_total FROM TopSpendersView WHERE order_total > 2000 ORDER BY order_total DESC;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ | Filter rows:

Extra options

username	order_total ▼ 1
mike_rock	3200.00
alex_music	2500.00